SISTEMAS DISTRIBUÍDOS · UFC

Aquário Plantado Inteligente

Os equipamentos reais de um aquário ligados a um Gateway por multicast, UDP, gRPC e TCP.

[Nomes e matrículas do grupo]

Prof. Dr. Paulo A. L. Rego · [data da apresentação]

O PROBLEMA

Luz, CO2 e temperatura precisam andar juntos

Luz demais ou CO2 de menos

As plantas não conseguem usar a energia e as algas tomam conta.

CO2 demais

Os peixes sufocam. É o jeito mais comum de perder todos os peixes de uma vez.

Temperatura errada

Peixes tropicais adoecem, e a água quente segura menos oxigênio.

Hoje cada equipamento tem o próprio timer e o próprio app. Se o CO2 trava ligado, ninguém fica sabendo, e não há histórico para ajustar o aquário depois.

A SOLUÇÃO

Um Gateway central que enxerga todos os equipamentos, guarda o histórico e reage na hora quando algo sai do normal.

3

equipamentos reais: a luminária Smartled e duas tomadas Wi-Fi

2

sensores simulados, de pH e de temperatura

1

arquivo .proto com todas as mensagens do sistema

AS FONTES DE DADOS

Cinco processos, um por equipamento

Fonte	Equipamento	Origem	Comando gRPC
luminaria-smartled	Smartled, controlador Magic Home 0x41	real	porta 50051
tomada-co2	Tomada Gaya 9812 (Tuya) que liga o CO2	real	porta 50052
tomada-luminaria	Tomada Gaya 9812 da luminária secundária	real	porta 50053
sensor-ph	Modelo que acompanha a tomada do CO2	simulado	não aceita
sensor-temperatura	Oscilação em torno de 26 °C	simulado	não aceita

As tomadas medem potência, corrente e tensão. CO2 ligado com 0 W indica defeito no equipamento, mesmo com a tomada ligada.

ARQUITETURA

Quem fala com quem, e por onde



Verde: equipamentos reais · tracejado: simulados

DESCOBERTA · UDP MULTICAST

Quem está na rede?

1. O Gateway manda um DiscoveryRequest para 239.10.10.10:50000 ao subir e repete a cada 30 s.
2. Cada fonte está inscrita no grupo e responde por unicast na porta 50001 com id, tipo, IP, porta gRPC e estado.
3. Com o endereço do Gateway em mãos, a fonte passa a mandar leituras para a porta 50002.

O parâmetro --ip-local escolhe a interface de rede do multicast. Com a VPN ligada, o pacote saía pela interface errada.

```
message DiscoveryRequest {
  string gateway_ip = 1;
  uint32 porta_resposta = 2; // 50001
  uint32 porta_dados = 3;    // 50002
}
message DiscoveryResponse {
  string fonte_id = 1;
  TipoFonte tipo = 2;
  string ip = 3;
  uint32 porta_grpc = 4;
  bool controlavel = 5;
  Status estado = 6;
}
```

FLUXO DE DADOS · UDP

Leituras periódicas, eventos na hora

```
message Medida {
  string nome = 1;    // "ph", "potencia"
  double valor = 2;
  string unidade = 3; // "pH", "W"
}
message Leitura {
  string fonte_id = 1;
  TipoFonte tipo = 2;
  int64 timestamp_ms = 3;
  repeated Medida medidas = 4;
  bool evento = 5;
  string motivo_evento = 6;
}
```

A fonte lê o aparelho a cada 1 ou 2 s e envia uma Leitura a cada 10 s. Um evento sai na hora, com evento = true:

motivo_evento	quando
ph_baixo	pH abaixo de 6,4
temperatura_alta	acima de 30 °C
mudou_estado	alguém mexeu pelo app
ligada_sem_consumo	menos de 0,5 W por 60 s

Medida é genérica, então uma fonte nova não muda o .proto.

COMANDOS · GRPC

Cada fonte controlável é um servidor gRPC

```
service FonteControlavel {
  rpc GetStatus(Vazio) returns (Status);
  rpc SetPower(ComandoPower) returns (Status);
  rpc SetParametro(ComandoParametro)
      returns (Status);
  rpc SimularFalha(ComandoFalha)
      returns (Status);
}
// todas:     intervalo_envio_s
// luminária: intensidade (0-100), rampa_s
```

O Gateway é o cliente e toda chamada tem prazo de 3 s.

Parâmetro desconhecido volta como INVALID_ARGUMENT. Fonte fora do ar dá UNAVAILABLE, que o cliente mostra como “fonte inacessível”.

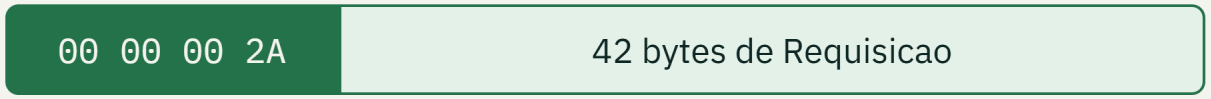
SimularFalha faz a fonte parar de enviar leituras sem encerrar o processo.

CLIENTE E GATEWAY · TCP

Uma requisição, quatro tipos de pedido

```
message Requisicao {
  oneof tipo {
    ListarFontes listar = 1;
    Comando comando = 2;
    Consulta consulta = 3;
    PedidoHistorico historico = 4;
  }
}
message Resposta {
  bool ok = 1;
  string erro = 2;
  oneof corpo { ... }
}
```

TCP entrega um fluxo de bytes, sem marcar onde uma mensagem termina. Por isso cada mensagem vai precedida do seu tamanho em 4 bytes.



O CLI em Python e o dashboard em Next.js usam o mesmo protocolo. No dashboard, as rotas de API do servidor abrem a conexão com o Gateway.

ADAPTADORES

O Gateway não sabe o que é Tuya nem Magic Home

Fonte	Protocolo por baixo	Biblioteca
Smartled	Magic Home por TCP 5577, sem chave; brilho no canal warm white	flux_led
Tomadas Gaya	Tuya v3.3 por TCP 6668, criptografado com a local key	tinytuya
pH e temperatura	Modelo próprio; o pH segue o estado da tomada do CO2	nenhuma

Cada fonte esconde o protocolo do fabricante. Trocar um aparelho real por um simulado é mudar o parâmetro --backend, e Gateway e cliente continuam iguais.

CONSULTAS E PERSISTÊNCIA

O que dá para perguntar ao Gateway

`consumo`

Energia em Wh de cada tomada, integrando a potência pelo método dos trapézios.

`tempo-ligado`

Horas que cada fonte controlável passou ligada.

`ph`

O menor pH da janela e quantos alertas de pH houve.

`media`

Média, desvio padrão, mínimo e máximo de uma medida de uma fonte.

`variacao`

A medida com o maior coeficiente de variação.

`historico`

Série de uma medida para o gráfico, reduzida a poucos pontos.

A janela padrão é de 60 min. Cada leitura é gravada com o horário de chegada no Gateway.

SEGURANÇA DO AQUÁRIO

Duas proteções contra excesso de CO2

pH < 6,4

O Gateway desliga o CO2

O sensor manda ph_baixo na hora e o Gateway chama SetPower(false) na tomada-co2. No teste com a tomada real, o CO2 desligou no mesmo segundo do alerta.

90 s

Sem Gateway, a tomada se desliga

A tomada-co2 conta o tempo desde o último pedido de descoberta. Se o Gateway cair com o CO2 ligado, ela desliga sozinha.

FALHAS

Quando uma parte cai

Situação	O que acontece
Processo de uma fonte encerrado	Vira OFFLINE depois de 3 intervalos sem leitura (cerca de 32 s no teste) e volta a ONLINE quando reinicia
falha <id> on	A fonte para de enviar leituras, continua viva e informa em_falha
Comando para fonte fora do ar	O cliente recebe “fonte inacessível (UNAVAILABLE)” e nada trava
Gateway reiniciado	Redescobre as cinco fontes no primeiro pedido de descoberta
CLI perde a conexão	Reconecta e repete o pedido uma vez

TECNOLOGIAS

Três linguagens, um .proto

Gateway

Java 21

- grpc-java
- protobuf-java
- sqlite-jdbc
- Maven

Fontes e cliente CLI

Python 3.11

- grpcio e protobuf
- tinytuya (tomadas)
- flux_led (Smartled)

Dashboard

TypeScript

- Next.js
- protobufjs
- SWR e Recharts

As três partes leem o mesmo proto/aquario.proto, então uma mensagem nova ou alterada é escrita em um lugar só.

TESTES

Simulado primeiro, depois o aquário

13

testes automatizados passando: 5 em Python, 6 em JUnit e 2 no dashboard

12

cenários de integração com os aparelhos simulados, incluindo as falhas

9

verificações no hardware real, com as tomadas e a Smartled

Ficaram de fora do teste real o failsafe e o reinício do Gateway com o CO2, para não mexer no timer do aquário. Os dois passaram no simulado, que roda o mesmo código.

DIFICULDADES

O que deu errado e como resolvemos

Problema	Causa	Correção
Multicast não chegava	VPN e várias interfaces de rede	Parâmetro --ip-local obrigatório
Comandos gRPC davam UNAVAILABLE	O garbage collector do Python recolhia o servidor gRPC	A fonte guarda a referência
Alarme falso de tomada sem consumo	A tomada Tuya leva ~30 s para atualizar a potência	Espera de 60 s
Tomadas exigem chave	O protocolo Tuya é criptografado	Chaves locais, fora do git
Slider só enviava o 1º valor	Estado antigo do React	Lê o valor do próprio elemento

OBRIGADO

Os aparelhos do aquário agora respondem a um Gateway, e o CO2 tem duas proteções.

github.com/jandernunes/trabalhosd2

Vídeo: [link do YouTube]

Perguntas?